

RTTY in Lyra — fldigi port plan

Status: design lock from operator discussion 2026-09-27. No code yet.

Home: this repo (Lyra-P2), same Cursor workspace as CW. Not a second project and not a second GitHub repo.

License: Lyra is GPL v3-or-later. fldigi is GPL v3-or-later. Same posture as the CW receive port (src/dsp/cw_fldigi/, NOTICE.md).

This document is the run-through: what we are building, what we reuse, what we do not build, and the staged work.

0. Why this exists

HF digital QSOs in the log are dominated by FT8, then FT4. After those (and beside CW as a separate world), **RTTY is still the main keyboard / HF contest digital mode** (ARRL RTTY Roundup, CQ WPX RTTY, NAQP RTTY, BARTG, and similar). Casual RTTY chat is smaller than it was; contests and some DX keep it alive.

FT8/FT4 stay in **MSHV / WSJT-X** over TCI + VAC. They are not fldigi modes and will not be ported from fldigi.

Lyra already:

- Has **DIGU / DIGL** and TX-rack bypass so digital audio is not voice-shaped (PHROT / EQ / compressor / combinator / etc. off).
- Feeds **fldigi / MSHV / WSJT-X** over TCI and VAC today. Companion

RTTY already works if the operator uses fldigi.

- Has a proven **in-radio decoder pattern**: fldigi CW receive math in

src/dsp/cw_fldigi/, Lyra glue in CwDecoder, audio tap in WdspEngine (post-demod 48 kHz, mode-gated), panel CwDecoderPanel.qml.

Native RTTY is for operators who want **copy and (later) transmit inside Lyra** without a second program — same reason the CW decoder exists.

1. Locked decisions

Topic	Lock
Pattern	Faithful fldigi modem + Lyra-native glue (resample, UI, PTT, DIGU). No Lyra-invented Baudot decoder.
Repo / Cursor	Same tree, same workspace. Feature branch off lyra-p2 when coding starts.
License	GPL v3+. Keep W1HKJ (and any other names in the fldigi files) in headers + NOTICE.md.
Sideband	USB / DIGU on all HF bands , including 80 m and 40 m. Voice LSB convention does not apply.
Reverse	Operator Rev toggle (invert mark/space or DIGL) for upside-down stations. Default: not reversed.
TX processing	None . Tones only. DIGU rack bypass already exists; native TX must use it (or later IQ FSK that never enters the mic chain).
Modulation	AFSK into DIGU as the portable path for HL2+, Brick, ANAN (all TX I/Q over Ethernet). Optional later: IQ FSK (same shift in baseband I/Q). Never bit-bang the TX NCO at 45 baud. Never use HL2 CW I-sample bits.
Hardware FSK jack	Out of scope (not common across HL2 / Brick / ANAN).
Build order	RX first (like CW). TX AFSK second. IQ FSK only if RX+AFSK is solid.

Topic	Lock
Audio tap	Same family as CW: in-process post-demod RX audio at 48 kHz. Gate to DIGU/DIGL (and/or an explicit “RTTY decode” on) so the tap is free when unused.
UI	Own floating panel + header chip (sibling of CW Dec), not stuffed into the CW panel. Panadapter stays the tuner.
Defaults (contest)	45.45 baud, 170 Hz shift, Baudot (ITA2), unshift-on-space as fldigi default unless we match a documented contest convention in a later pass. High tones 2125 / 2295 Hz unless fldigi's RTTY defaults differ — verify in fldigi 4.2.x source before coding , do not guess.
Combo / SDRLogger+	Yes — same Combo as CW. Grab call, shared contact row, {LOG} → existing lyra_contact / lyra_log on the TCI socket. No second protocol. Stamp RTTY on the log (not DIGU). Full transcript is not streamed to the logger (CW does not either).
Out of scope (this project)	FT8/FT4, PSK31 (possible later), FreeDV/RADE, packet, AMTOR. Logging is in scope via Combo, not a built-in logger.

2. What “AFSK vs FSK” means here

- **FSK** = mark and space as two RF frequencies (170 Hz apart).
- **AFSK** = two **audio** tones into SSB. On USB, that is 170 Hz RF

FSK on the air.

For Lyra/HL2+/Brick/ANAN the radio never has a shared hardware FSK pin. AFSK on DIGU (processors off) is the correct first TX. IQ FSK is the same on-air signal generated in I/Q instead of through the mic/SSB audio stage — cleaner, still one code path for all three families.

3. Architecture (mirror CW)

RX: HL2/Brick/ANAN IQ
 -> WDSP RXA (DIGU)
 -> 48 kHz mono tap (WdspEngine, RTTY-gated)
 -> RttyDecoder adapter (decimate to fldigi RTTY rate)
 -> lyra::dsp::rttyfldigi (faithful port)
 -> onText callback -> Qt signal -> RttyDecoderPanel

TX (phase 2): keyboard / macros
 -> Baudot + AFSK tone generator (fldigi TX or Lyra-native tones)
 -> DIGU TXA with rack bypass (tones only)
 -> EP2 / P2 I/Q -> radio

3.1 New files (proposed)

Piece	Path
fldigi RTTY modem port	src/dsp/rtty_fldigi/ (fldigi_rtty.{h,cpp} + filters reused or shared with cw_fldigi if identical)
Lyra adapter	src/dsp/RttyDecoder.{h,cpp}
TX tone gen (phase 2)	src/dsp/RttyModulator.{h,cpp} or TX side of the same fldigi port
Panel	src/qml/RttyDecoderPanel.qml
Attribution	NOTICE.md section sibling to fldigi CW
Tests	tests/dsp/ synthetic 45.45 baud AFSK -> known Baudot string

Reuse: `fftfilt / gfft / moving average` if `fldigi` RTTY uses the same helpers as CW. Do not duplicate three copies; share or include.

3.2 Existing hooks to reuse (verify at code time, do not invent)

- `WdspEngine` post-demod tap used by `cwDecoder_.process(...)` (~audio thread, ~48 kHz). Add a parallel RTTY gate (`rttyModeActive_ / rttyDecodeOn_`) on DIGU/DIGL.
- TX rack bypass: `tx_rack_bypass / DIGU-DIGL` already skips native voice rack. Confirm native RTTY TX goes through that path.
- Mode: set **DIGU** when the operator opens RTTY (do not auto-LSB on 40/80).
- Dock pattern: `MainWindow` loads `CwDecoderPanel.qml` — clone for RTTY chip + panel.
- `QSettings: rtty/*` standalone (not TX profile), like `cw/decoder*`.
- Combo: `TciServer + CwMacroModel` (`hisCall / rst / opName / logQsoRequested`). RTTY grab writes the **same** His Call. `{LOG}` from an RTTY send uses the same `lyra_log` path with `mode=RTTY`.

3.3 fldigi source to port

Upstream `fldigi` (verify version at port time; CW used **4.2.06**):

- `src/cw_rtty/rtty.cxx, rtty.h` (modem)
- Shared filters as actually called by RTTY (`fftfilt`, etc.)
- Baudot tables in `fldigi` (do not invent ITA2)

Strip `FLTK / progdefaults` globals → plain members + setters (same as `fldigi_cw.h`). Receive path only in phase 1.

Need at implement time: a local copy of those `fldigi` files (SourceForge or git) for side-by-side; port with no algorithmic drift.

4. Operator run-through (when it ships)

Receive

1. Tune the RTTY signal on the panadapter (typical HF RTTY is USB).
2. Mode **DIGU**. Filter wide enough for 170 Hz + tone pair (often ~300–500 Hz or `fldigi`'s RTTY bandwidth — match `fldigi` at implement time).
3. Open **RTTY** panel, enable decode.
4. If garbage letters: hit **Rev** (inverted station or accidental LSB).
5. Set baud 45.45 (contest default); 50/75 only when the other station uses them.

Transmit (phase 2)

1. Stay DIGU. Confirm TX processors stay off.
2. Type in the RTTY pane (or a small macro row: CQ / DE / K — contest extras later).
3. PTT via existing MOX / footswitch / TCI; AFSK tones only.
4. Dummy-load first: other SDR or `fldigi` should copy clean Baudot.
5. If Combo is on: His Call in the shared contact row is already in `SDRLogger+`; `{LOG}` on a RTTY macro (or a Log button) writes the QSO as **RTTY** at the current carrier.

Companion path (unchanged)

fldigi over VAC/TCI still works. Native RTTY is optional, not a replacement for MSHV FT8.

5. Staged work

Phase 0 — Verify (half day)

- Open fldigi 4.2.x `rtty.cxx`: sample rate, tone pair, baud defaults, Baudot, USB vs reverse.
- Confirm Lyra DIGU TX bypass is complete for a native tone injector (same as TCI digital).
- Confirm RX tap sample format (float, interleaved vs mono) at the CW hook.

Phase 1 — RX only (the CW-shaped project)

1. Port fldigi RTTY **receive** into `src/dsp/rtty_fldigi/`.
2. `RttyDecoder` adapter: 48 kHz → modem rate.
3. Wire tap in `wdspEngine`; DIGU/DIGL + enable gate; reset on mode edge.
4. `RttyDecoderPanel.qml`: transcript, baud, shift, Rev, squelch if fldigi exposes it, clear.
5. Unit tests: synthetic AFSK → expected text.
6. NOTICE.md + help one-pager later (not this PDF's job).
7. Bench: 40 m / 80 m **DIGU** (prove LSB-is-wrong), 20 m contest snippet if on air.
8. Combo (same toggle as CW): right-click grab → His Call → `lyra_contact` with Combo on and SDRLogger+ on TCI. Prove callbook name-back still fills {NAME}.

Done when: dummy-load or off-air recording copies known text; Rev works; 80/40 default USB; grab-to-logger matches CW.

Phase 2 — TX AFSK (tones only)

1. fldigi RTTY TX or a tiny AFSK oscillator driven by the same Baudot table (prefer fldigi TX if it is cleanly separable).
2. Inject into DIGU TX path **after** rack bypass (or as the digital source). No EQ/comp/PHROT/combinator.
3. Keyboard + PTT; optional tiny CQ / DE / K macros.
4. {LOG} (or Log) on RTTY send → `TciServer::onLogQsoRequested` with **mode forced to RTTY** (do not send DIGU into the ADIF). RST stays the shared contact RST (typically 599).
5. Bench: second receiver / fldigi copies Lyra TX; Palstar/dummy; no voice-processor artifacts. Combo: {LOG} appears in SDRLogger+ as RTTY, not CW.

Done when: other end copies; spectrum shows two steady tones 170 Hz apart, not a processed blob; logger mode is RTTY.

Phase 3 — optional IQ FSK

Generate mark/space as baseband I/Q (skip mic). Same operator UI. Only if phase 2 is limited by the audio/SSB stage. Still not NCO FSK.

6. UI sketch (phase 1)

Header chip: **RTTY** (next to CW Dec).

Panel:

- Scrolling monospace transcript + Clear
- **On** decode
- Baud: 45.45 / 50 / 75
- Shift: 170 (and fldigi’s other shifts if we expose them)
- **Rev**
- Status: USB/DIGU reminder (“RTTY is USB on all HF bands”)
- Right-click grab: **His Call / Name** → same CwMacros as CW Dec
- Phase 2: TX text line + TX enable + macros including {LOG}

No second waterfall. No built-in logger — Combo is the logger.

7. SDRLogger+ Combo (same link as CW)

CW already shares one contact with SDRLogger+ over TCI. Native RTTY must use **that** link, not a new socket and not a live Baudot dump.

What CW does today (do not reinvent):

Step	Mechanism
Combo on	Settings → Network → “SDRLogger+ Combo...” → lyra_combo:on
Call out	Decoder grab or His Call field → CwMacroModel::contactChanged → lyra_contact:lyra,, ...
Name back	SDRLogger+ callbook → lyra_contact:sdrlog, ... → {NAME}
RST S	Combo lyra_snr + RX1 meter (already works on digital)
Log	{LOG} in a macro → lyra_log:,,,,,

RTTY mapping:

1. **One contact row.** RTTY grab writes CwMacros.hisCall (and Name the same way). Contest serial / RST stay on that model. The operator does not keep two His Call boxes.
2. **Same Combo checkbox.** RTTY does not get a second master toggle.
Relabel later to “share Lyra contact” if the CW-only wording confuses people — optional polish, not a new flag.
3. **Log mode = RTTY.** onLogQsoRequested today sends prefs_->mode(), which would be **DIGU** during AFSK. Contest ADIF wants **RTTY**. Phase 2: if the send came from the RTTY panel / RTTY {LOG}, stamp RTTY. Echo guard and Combo-off gates stay identical.
4. **Do not stream the transcript.** CW Combo does not push every decoded letter. Grab + log is enough. A live lyra_rtty: tape would be a new SL+ feature — out of scope unless asked.
5. **SDRLogger+ code.** Protocol is already lyra_*. Confirm SL+ ADIF accepts mode RTTY on lyra_log (likely yes). Any Combo UI that says “CW only” needs a one-line copy/mode pass in the logger repo — Lyra-side work is reuse, not a parallel Combo.

Operator run-through (Combo): Combo on, TCI connected, decode RTTY, right-click the call → His Call → logger entry + QRZ. Finish with {LOG} (phase 2) or log in SDRLogger+ by hand after phase 1 grab.

Canonical Combo spec: docs/architecture/combo_link_design.md.

8. Risks and discipline

- **Wrong sideband on 40/80** — default DIGU; document in the panel.
 - **TX processing** — if native TX accidentally hits the voice rack, the other end's decoder dies. Gate with tests / mode flag.
 - **GIL is gone; keep the audio thread light** — fldigi RTTY is cheap vs CW neural; still no Qt in the modem process().
 - **fldigi GUI globals** — porting trap; CW already solved this.
 - **Do not ship TX before RX copies** — same CW lesson.
 - **Contest unshift-on-space / figs** — copy fldigi; do not guess US vs EU figs.
 - **DIGU vs RTTY in the log** — if {LOG} forgets to stamp RTTY, the QSO is wrong in ADIF even when on-air was correct.
-

9. What we need before coding

1. Operator go-ahead to start Phase 0/1 on a feature branch.
 2. fldigi 4.2.x source tree available locally for the port (same as CW).
 3. Dummy load + a second decoder (fldigi or another SDR) for TX bench when phase 2 starts.
 4. No WSL/Linux required for this feature.
 5. For Combo: SDRLogger+ already on TCI (same as CW). Phase 1 only needs grab → His Call. Phase 2 needs a logger bench that {LOG} stores RTTY. Keep the SDRLogger+ repo handy if ADIF mode needs a one-line accept.
-

10. Suggested first commit series (when greenlit)

1. dsp: fldigi RTTY receive port (no UI) + tests
2. dsp: RttyDecoder adapter + WdspEngine tap
3. ui: RttyDecoderPanel + header chip + grab → CwMacros (Combo for free)
4. (later) tx: RTTY AFSK into DIGU, rack bypass
5. (later) combo: RTTY {LOG} stamps mode RTTY

Co-Authored-By: Cursor, as usual. Commit only when asked.

Locked 2026-09-27 — N8SDR: native RTTY like CW; fldigi GPL v3+; DIGU all bands; tones only; AFSK then optional IQ FSK; Combo same as CW (lyra_contact / lyra_log, mode RTTY); FT8 stays MSHV.